PROGRAMMING FUNDAMENTALS

Spring 2024

16th May

File

- When a program needs to save data for later use, it writes the data in a file
- The data can then be read from the file at a later time

Why files?

- Regular programs require the user to reenter data each time the program runs
- Data is kept in variables in RAM and disappears once the program stops running
- To retain data between the times it runs, data is saved in a file, which is usually stored on a computer's disk.
- A file remains there after the program stops running

An output file

- An *output file* is a file that data is written to
- The process of saving data in a file is *writing data* to the file
- When a piece of data is written to a file, it is copied from a variable in RAM to the file
- It is called an output file because the program stores output in it

An Input file

- An *input file* is a file that data is read from
- It is called an input file because the program gets input from the file
- The process of retrieving data from a file is known as *reading data* from the file
- When a piece of data is read from a file, it is copied from the file into a variable in RAM

Working with a file

1.Open the file— Opening a file creates a connection between the file and the program

- Opening an output file usually creates the file on the disk and allows the program to write data to it
- Opening an input file allows the program to read data from the file

2. Process the file— Data is either written to the file (if it is an output file) or read from the file (if it is an input file)

3. Close the file— After the program is finished using the file, the file must be closed

Closing a file disconnects the file from the program

Types of files

- 1. A text file contains data that has been encoded as text, using a scheme such as ASCII or Unicode
  - A text file may be opened and viewed in a text editor such as Notepad
- 2. A binary file contains data that has not been converted to text
  - You cannot view the contents of a binary file with a text editor

File Access Methods

- Two ways to access data stored in a file: sequential access and direct access

1. In a *sequential access file* , you access data from the beginning of the file to the end of the file

- If you want to read a piece of data that is stored at the very end of the file, you have to read all of the data that comes before it—you cannot jump directly to the desired data

File Access Methods

- 2. With a *random access file* (also known as a *direct access file*) you can jump directly to any piece of data in the file without reading the data that comes before it

File stream object

- In order for a program to work with a file on the computer's disk, the program must create a file stream object in memory
- A *file stream object* is an object that is associated with a specific file and provides a way for the program to work with that file
- It is called a "stream" object because a file can be thought of as a stream of data

File stream object

- File stream objects work very much like the cin and cout objects
- A stream of data may be read from the keyboard by cin, and stored in variables
- Streams of data may be sent to a file stream object, which writes the data to a file
- When data is read from a file, the data flows from the file stream object that is associated with the file, into variables

fstream Object

- The fstream header file defines the data types ofstream, ifstream , and fstream
- C++ program can work with a file only when it defines an object of one of these data types.
- The object will be “linked” with an actual file on the computer’s disk

ofstream

- Output file stream
- You create an object of this data type when you want to create a file and write data to it.

ifstream

- Input file stream
- You create an object of this data type when you want to open an existing file and read data from it

fstream

- File stream
- Objects of this data type can be used to open files for reading, writing, or both

File I/O

- Just as cin and cout require the iostream file to be included in the program
- The file fstream contains all the declarations necessary for file operations

```
#include <fstream>
```

Creating a file object

- Before data can be written to or read from a file, the following things must happen:
- A file stream object must be created
- The file must be opened and linked to the file stream object

Opening a file for input (reading)

```
ifstream inputFile;
inputFile.open("Customers.txt");
```

- The first statement defines an ifstream object named inputFile
- The second statement calls the object's open member function, passing the string "Customers.txt" as an argument
- The open member function opens the Customers.txt file and links it with the inputFile object
- Now you can use the inputFile object to read data from the Customers.txt file

Opening a file for output (writing)

```
ofstream outputFile;
outputFile.open("Employees.txt");
```

- The first statement defines an ofstream object named outputFile
- The second statement calls the object's open member function, passing the string "Employees.txt" as an argument
- The open member function creates the Employees.txt file and links it with the outputFile object
- Now you can use the outputFile object to write data to the Employees.txt file
- If the specified file already exists, it will be erased, and a new file with the same name will be created

An alternate way

- It is possible to define a file stream object and open a file in one statement

```
ifstream inputFile("Customers.txt");

ofstream outputFile("Employees.txt");
```

Closing a file

- Although a program's files are automatically closed when the program shuts down
- It is a good programming practice to write statements that close them

Why Close?

- Most operating systems temporarily store data in a *file buffer* before writing to a file to improve the system's performance
- A file buffer is a small "holding section" of memory that file-bound data is first written to
- When the buffer is filled, all the data stored there is written to the file
- Closing a file causes any unsaved data that may still be held in a buffer to be saved to its file

Why Close?

- Some operating systems limit the number of files that may be open at one time
- When a program closes files that are no longer being used it is better practice to close them to save operating system's resources

```
inputFile.close();
```

Writing data to a file

- The stream insertion operator (<<) used with the cout object writes data to the screen
- It can also be used with ofstream objects to write data to a file

```
ofstream outputFile;
outputFile.open("demofile.txt");

outputFile << "Bilal\n";
```

the statement looks like a `cout` statement, except the name of the `ofstream` object name replaces `cout`

Example

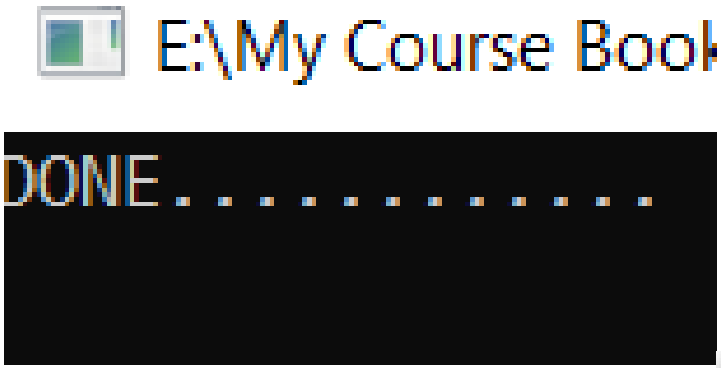
```
outputFile << "Price: " << price << endl;
```

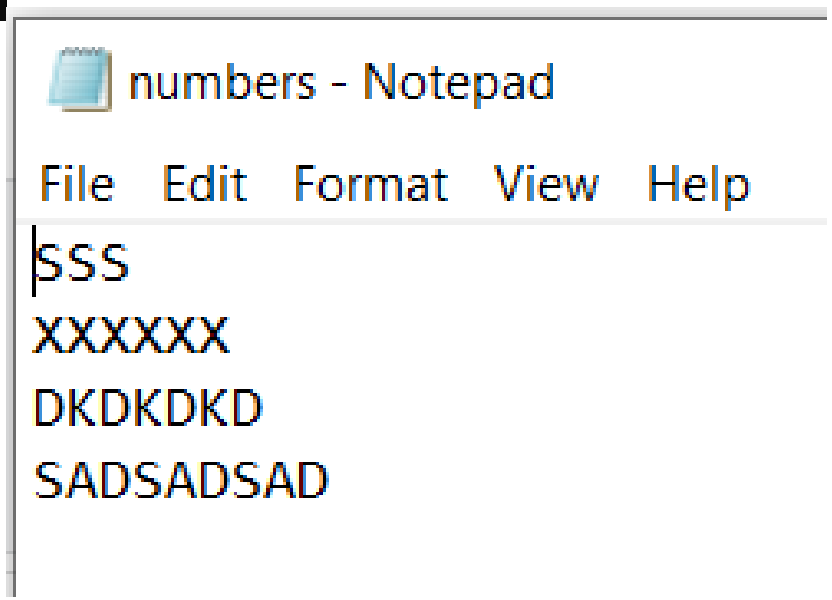
Code

```
// This program reads data from a file.
#include <iostream>
#include <fstream>
using namespace std;
int main()
 {
 ofstream outputFile;

 outputFile.open("E:\\TextFiles\\numbers.txt");
 //now writing data to the file
 outputFile << "SSS\n";
outputFile << "XXXXXX\n";
outputFile << "DKDKDKD\n";
outputFile << "SADSADSAD\n";
outputFile.close();
cout<< "DONE.............\n";
 return 0;
 }
```

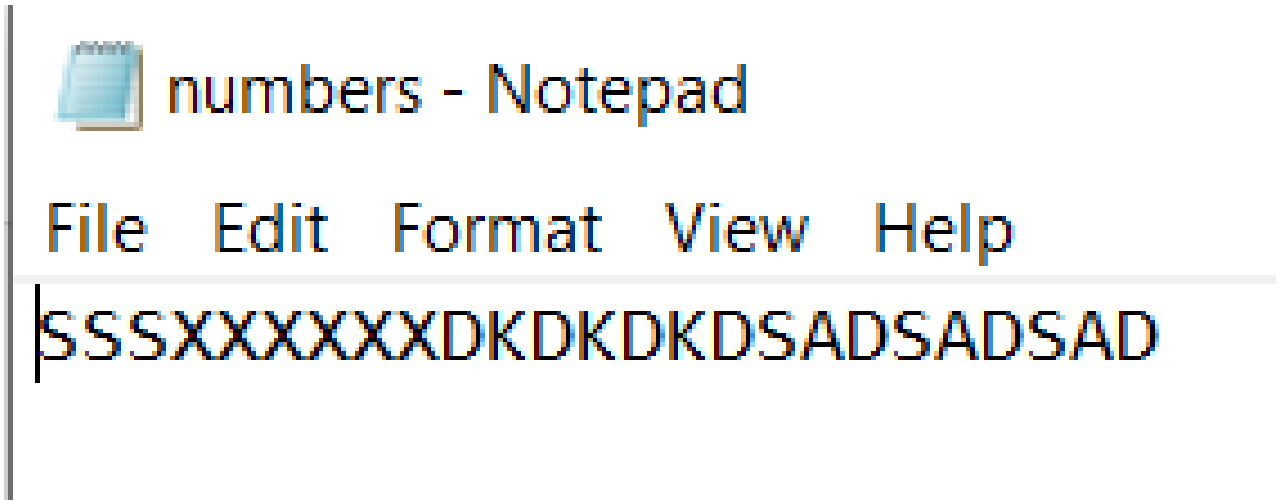
Output


E:\My Course Book
DONE . . . . . .


numbers - Notepad
File Edit Format View Help
SSS
XXXXXX
DKDKDKD
SADSADSAD

Practice-1

- Remove the \n character and see the contents of the file



Practice-2

- Get ten integers from the user in the main program using a loop and write them into a file

P.S.

- When opening a file, you will need to specify its path as well as its name
- The following statement opens the file C:\data\test_file.txt :

```
inputFile.open("C:\\data\\test_file.txt")
```

- In this statement, the file C:\data\test_file.txt is opened and linked with inputFile

Example

- Note the use of two backslashes in the file's path
- Two backslashes are needed to represent one backslash in a string literal

```
ofstream outputFile;
outputFile.open("D:\\txt-Files\\demofile.txt");
cout << "Now writing data to the file.\n";
```

Reading data from file

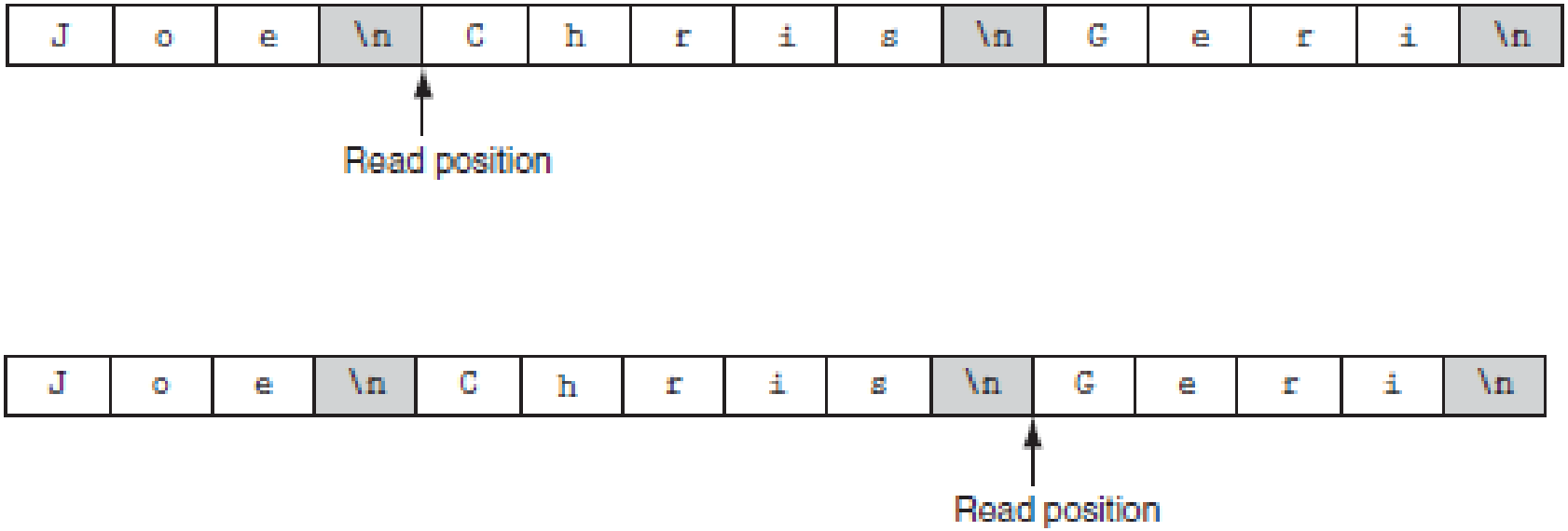
- The >> operator not only reads user input from the cin object
- The >> operator also reads data from a file
- Consider **inputFile** is an ifstream object

```
inputFile >> name;
```

Read Position

- When a file has been opened for input, the file stream object internally maintains a special value known as a *read position*
- A file's read position marks the location of the next byte that will be read from the file
- When an input file is opened, its read position is initially set to the first byte in the file
- The first read operation extracts data starting at the first byte
- As data is read from the file, the read position moves forward, towards the end of the file

Read Position



Reading numeric data

- When data is stored in a text file, it is encoded as text, using a scheme such as ASCII or Unicode
- The numbers are also stored in the file as a series of characters
- The numbers are stored in the file as the strings "10", "20", and "30"
- You can use the >> operator to read numeric data from a text file, into a numeric variable
- The >> operator will automatically convert the data to a numeric data type

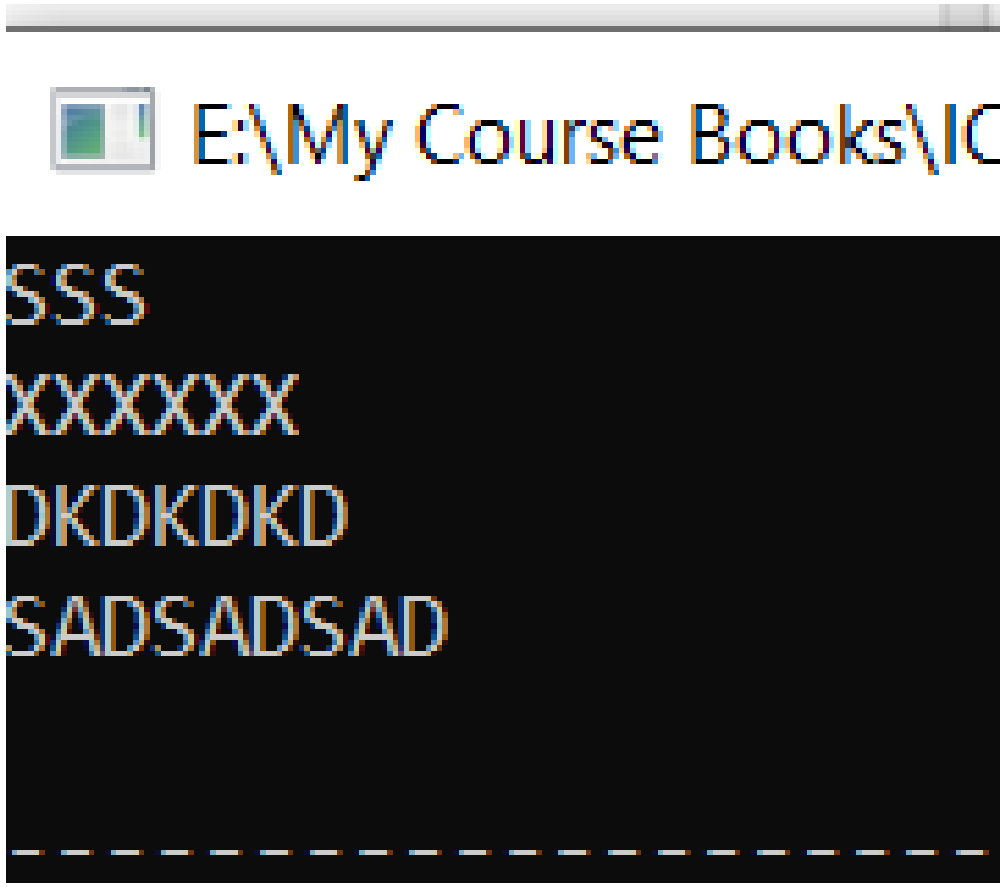
Detecting End of File (EoF)

- Suppose you need to write a program that displays all items in a file, but you do not know how many items the file contains
- You can open the file and then use a loop to read an item from the file and display it
- An error will occur if the program attempts to read beyond the end of the file
- The >> operator not only reads data from a file, but also returns a true or false value indicating whether the data was successfully read or not
- If the operator returns true, then a value was successfully read
- If the operator returns false, it means that no value was read from the file

Example-1

Q1.cpp | Lec-21.cpp | Lec-21-Q1.cpp

```cpp
// This program reads data from a file.
#include <iostream>
#include <fstream>
using namespace std;
int main()
 {
 ifstream MyFile;
 string name;
 MyFile.open("E:\\TextFiles\\numbers.txt");
 while (MyFile >> name)
 {
 cout << name << endl;
 }
 MyFile.close();
 return 0;
 }
```

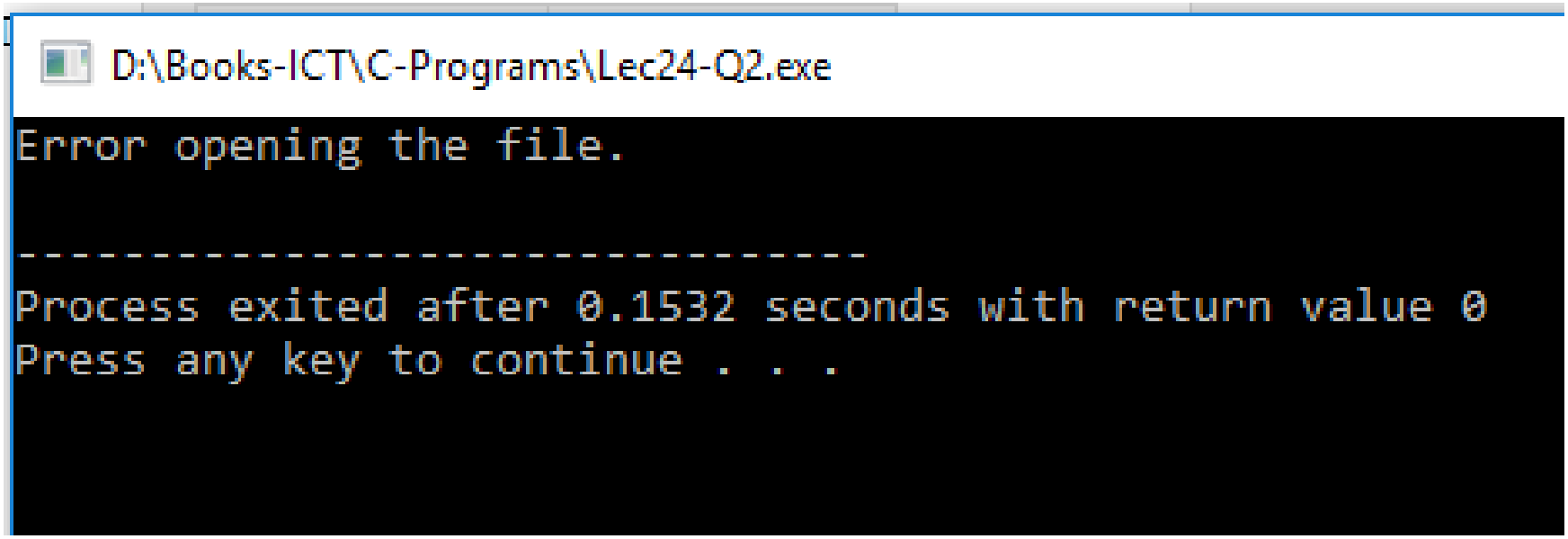

E:\My Course Books\IC
SSS
XXXXXX
DKDKDKD
SADSADSAD

EoF

- The statement in While loop that extracts data from the file is used as the Boolean expression
- The expression MyFile >> number executes
- If an item is successfully read from the file, the item is stored in the number variable and the expression returns true
- If there are no more items to read from the file, the expression MyFile >> number returns false, indicating that it did not read a value
- In that case, the loop terminates

Example-2

```
// This program tests for file open errors.
#include <iostream>
#include <fstream>
using namespace std;
 int main()
 {
 ifstream MyFile;
 int number;
 MyFile.open("D:\\txt-Files\\number.txt");
// If the file successfully opened, process it.
 if (MyFile)
 {
 while (MyFile >> number)
 {
 cout << number << endl;
 }
 MyFile.close();
 }
 else
 {
 cout << "Error opening the file.\n";
 }
 return 0;
 }
```


D:\Books-ICT\C-Programs\Lec24-Q2.exe
Error opening the file.

Process exited after 0.1532 seconds with return value 0
Press any key to continue . . .

File open fails

- Another way to detect a failed attempt to open a file is with the fail member function
- The fail member function returns true when an attempted file operation is unsuccessful
- If the file could not be opened, the user should be informed and appropriate action taken by the program

```
if (MyFile.fail())
{
cout << "Error opening file.\n";
}
else
{
// Process the file.
}
```

Task-1

- Write a program that stores 15 random numbers between 1 and 100 (inclusive) in a text file.

Task-2

- Read the file created in task-1 and find the maximum value stored in the file.
- Your main() function displays the maximum value

Book Ref

- Tony Gaddis
- CH-5